

Medical Diagnosis AID

This is a brief final_report of Medical Diagnosis AID ML project made for IFO-L Laboratorio di informatica course exam by Giulio Mazzanti.

1. Problem statement

The goal of this project is to build a simple ML system that estimates the likelihood of diabetes from a small set of clinical measurements.

The project was designed as a practical educational MVP.

The main objective was not to build a real diagnostic tool, but to develop a complete and defendable pipeline that includes:

- data analysis
- preprocessing
- model training and evaluation
- a simple web interface with Flask
- reproducible execution with Docker

This problem is a binary classification task because the target variable has two possible values:

- 0 = no diabetes
 - 1 = diabetes
-

2. Dataset description

The project uses the **PIMA Indians Diabetes Database**.

The dataset contains **768 rows** and **9 columns**:

- Pregnancies
- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI

- `DiabetesPedigreeFunction`
- `Age`
- `Outcome`

`Outcome` is the target variable, while the other columns are input features.

This dataset was chosen because it is:

- small and fast to analyze
 - fully numeric
 - simple to use in a Flask form
 - appropriate for classical machine learning with `scikit-learn`
-

3. EDA findings

The exploratory data analysis was carried out in `eda.ipynb`.

The main findings were:

- the dataset has no null values in the original CSV file
- however, some features contain **zero values that are medically improbable**
- the target variable `Outcome` is **not perfectly balanced**
- therefore, accuracy alone is not enough to evaluate the models

During EDA, a distinction was made between:

- **valid zeros**, such as `Pregnancies = 0` or `Outcome = 0`
- **improbable zeros**, especially in:
 - `Glucose`
 - `BloodPressure`
 - `SkinThickness`
 - `Insulin`
 - `BMI`

These values were considered improbable because a value of zero in those measurements is generally unrealistic in a clinical context and may represent missing or badly recorded data.

EDA also helped verify:

- the feature distributions
- the class distribution of `Outcome`

- the need for basic preprocessing before training
-

4. Preprocessing

The preprocessing pipeline was intentionally kept simple and suitable for an MVP.

The following decisions were applied:

1. `Outcome` was left unchanged because it is the target label.
2. `Pregnancies` was left unchanged because zero pregnancies is a valid value.
3. improbable zeros in `Glucose`, `BloodPressure`, `SkinThickness`, `Insulin`, and `BMI` were replaced with missing values.
4. These missing values were then imputed using the **median** of each feature.

After cleaning, the feature matrix `X` and the target vector `y` were created.

The train-test split was performed with:

- `test_size = 0.2`
- `random_state = 42`
- `stratify = y`

Using `stratify = y` was important to preserve the class distribution between training and test data.

5. Models and metrics

Two classification models were trained and compared:

- `LogisticRegression(max_iter=1000)` as a baseline model
- `RandomForestClassifier(random_state=42)` as a stronger non-linear model

The metrics used were:

- accuracy
- precision
- recall
- f1-score

Logistic Regression results

- accuracy: 0.701299
- precision: 0.586957
- recall: 0.500000
- f1-score: 0.540000

Random Forest results

- accuracy: 0.779221
- precision: 0.727273
- recall: 0.592593
- f1-score: 0.653061

The notebook also includes confusion matrices for both models.

Final model choice

The final model selected for the project is **RandomForestClassifier**.

This choice was made because it achieved better results than Logistic Regression on all the evaluation metrics used in the notebook.

For this MVP, Random Forest provided the best balance between performance and implementation simplicity.

The final trained model was then saved as:

- models/diabetes_model.joblib

6. Flask implementation

The project includes a small Flask application in `app.py`.

The app loads the saved model using `joblib` and exposes three main routes:

- `/` for the home page
- `/predict` for form-based predictions
- `/predict-json` for JSON-based predictions

HTML workflow

The home page contains a form where the user can enter the eight input features:

- Pregnancies

- Glucose
- BloodPressure
- SkinThickness
- Insulin
- BMI
- DiabetesPedigreeFunction
- Age

When the form is submitted:

1. Flask reads the values from the request
2. the values are converted to `float`
3. a NumPy array is created with the same feature order used during training
4. the model generates:
 - a binary prediction
 - a probability for class `1`
5. the user gets redirected to `result.html` to see the result

JSON workflow

The `/predict-json` endpoint accepts a POST request with JSON input and returns a JSON response with:

- `prediction`
- `probability`

This route is useful to demonstrate that the project can work both as a simple web app and as a minimal API.

7. Docker usage

To make the project reproducible, a `Dockerfile` was added.

The Docker image is based on:

- `python:3.12-slim`

The container workflow is:

1. set `/app` as working directory
2. copy `requirements.txt`

3. install dependencies with `pip`
4. copy the full project
5. expose port `5000`
6. start the app with `python app.py`

Example commands:

```
docker build -t medical-diagnosis-aid .  
docker run -p 5000:5000 medical-diagnosis-aid
```

8. Limitations and future work

This project has several limitations because is an MVP.

The main limitations are:

- the dataset is relatively small
- the problem is simplified to a binary prediction
- the model does not replace medical evaluation
- the web app performs a simple direct prediction without advanced validation logic
- preprocessing is basic and intentionally minimal

Possible future improvements include:

- hyperparameter tuning
- more detailed feature analysis
- calibration of probabilities
- stronger input validation in the Flask app
- better visual reporting of the model output

In conclusion, the project satisfies the main educational requirements of the course: it uses Python, `scikit-learn`, Flask, Git/GitHub-oriented structure, and Docker in a complete but manageable end-to-end workflow.